

Esercizi Lezione 6

Bogdan Delia

2024-11-10

```
indagine1 = read.delim("~/Downloads/indagine1.txt")
# summary(indagine1)
# head(indagine1)

# Quale è l'altezza media degli studenti che provengono dall' Umbria?
mean(indagine1$Altezza[indagine1$Regione == "Umbria"])

## [1] 168.74

# Calcolare la spesa complessiva per il parrucchiere degli studenti (maschi) diplomati in LS.
LS = indagine1$Diploma == "LS"
M = indagine1$Genere == "M"
sum(indagine1$Parrucchiere[LS & M])

## [1] 6428

# Sono mediamente più alti gli uomini o le donne del Centro?
CENTRO = indagine1$Regione == "Centro"
MASCHI = indagine1$Genere == "M"
FEMMINE = indagine1$Genere == "F"

which.max(c(mean(indagine1$Altezza[MASCHI & CENTRO]),
             mean(indagine1$Altezza[FEMMINE & CENTRO])))

## [1] 2

# sono più alte le femmine

# Quanti libri leggono in media gli studenti che hanno fatto il COM?
COM = indagine1$Diploma == "COM"
round(mean(indagine1$Libri[COM],0))

## [1] 10

# Dato il dataset Indagine1.txt, si costruisca la tabella di contingenza delle
# variabili Sesso e Parrucchiere, dividendo la variabile Parrucchiere in classi
# [0 - 20); [20 - 35) e [35 - 201).
Parrucchiere_classi = cut(indagine1$Parrucchiere, breaks = c(0, 20, 35, 201),
                          right = FALSE)
tab_contingenza = table(indagine1$Genere, Parrucchiere_classi)

# Fra gli studenti che spendono [15 - 35) euro dal parrucchiere,
# quant'è la percentuale di femmine?
studenti_1535 = subset(indagine1, Parrucchiere >= 15 & Parrucchiere < 35)
percentuali_genere = prop.table(table(studenti_1535$Genere)) * 100
```

```

round(percentuali_genere["F"], 2)

##      F
## 45.45

# Fra i maschi, qual è la percentuale di quelle che spendono più di 35 euro?
prop_tabella = prop.table(tab_contingenza, margin = 1) * 100
round(prop_tabella[2, 3], 2) # 83.86

## [1] 83.86

# Quant'è la proporzione di studenti (totali)
# che prendono più di 5 caffè al giorno?
studentitot = nrow(indagine1)
studenticaffe = sum(indagine1$Caffe > 5)
studenticaffe / studentitot

## [1] 0.84

# Tra le fumatrici, qual è il numero medio di caffè?
# E' minore o maggiore di quello dei fumatori (maschi)?
fumatrici = subset(indagine1, Genere == "F" & Fumatore == "Si")
mediacaffe_F = mean(fumatrici$Caffe) # 17

fumatori = subset(indagine1, Genere == "M" & Fumatore == "Si")
mediacaffe_M = mean(fumatori$Caffe)
mediacaffe_F > mediacaffe_M # è maggiore il consumo delle fumatrici donne

## [1] TRUE

```

ESERCIZIO PCA Bogdan Delia Anamaria

1. Descrizione del DATA SET

```
install.packages("ISLR")
library(ISLR)
?NCI60

str(NCI60)
List of 2
 $ data: num [1:64, 1:6830] 0.3 0.68 0.94 0.28 0.485 ...
  .. attr(*, "dimnames")=List of 2
   .. ..$ : chr [1:64] "V1" "V2" "V3" "V4" ...
   .. ..$ : chr [1:6830] "1" "2" "3" "4" ...
  $ labs: chr [1:64] "CNS" "CNS" "CNS" "RENAL" ...

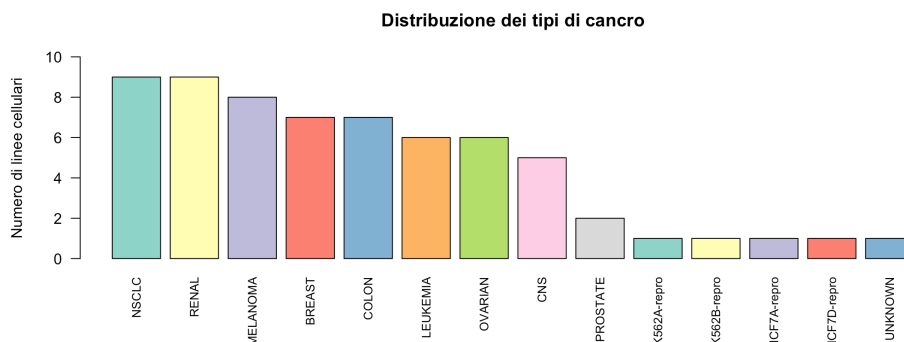
table(NCI60$labs)
      BREAST      CNS      COLON K562A-repro K562B-repro  LEUKEMIA
        7         5         7         1         1         6
MCF7A-repro MCF7D-repro  MELANOMA      NSCLC      OVARIAN  PROSTATE
        1         1         8         9         6         2
      RENAL      UNKNOWN
        9         1

library(RColorBrewer)
conteggio = sort(table(NCI60$labs), decreasing = TRUE)
colori = brewer.pal(n = min(9, length(cancer_counts)), name = "Set3")
```

Il dataset NCI60 contiene dati di espressione genica provenienti da 64 linee cellulari di cancro, con misurazioni su 6830 geni. Le 64 linee cellulari sono state sviluppate dal National Cancer Institute (NCI) e provengono da nove diversi tipi di cancro.

Ci sono 64 linee cellulari in totale, il tipo di cancro più rappresentato è NSCLC (tumore polmonare non a piccole cellule) e RENAL (tumore al rene), entrambi con 9 linee cellulari. Il tumore al seno (BREAST) ha 7 linee cellulari, ma ha anche due sottotipi particolari: MCF7A-repro e MCF7D-repro con una linea ciascuno. K562A-repro e K562B-repro sono due varianti della linea cellulare K562, che deriva da una leucemia mieloide cronica. UNKNOWN probabilmente significa che una linea cellulare non è stata classificata in un tipo specifico di cancro.

```
barplot(conteggio,
        ylim = c(0,10),
        las = 2,
        col = colori,
        main = "Distribuzione dei tipi di cancro",
        xlab = " ",
        ylab = "Numero di linee cellulari",
        cex.names = 0.8)
```



```
dataset = NCI60$data
dim(dataset)
[1] 64 6830

colnames(dataset)      # Nomi delle colonne (geni)
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9"
[10] "10" "11" "12" "13" "14" "15" "16" "17" "18"
[19] "19" "20" "21" "22" "23" "24" "25" "26" "27"
...
rownames(dataset)      # Nomi delle righe (linee cellulari)
[1] "V1" "V2" "V3" "V4" "V5" "V6" "V7" "V8" "V9" "V10" "V11"
[12] "V12" "V13" "V14" "V15" "V16" "V17" "V18" "V19" "V20" "V21" "V22"
[23] "V23" "V24" "V25" "V26" "V27" "V28" "V29" "V30" "V31" "V32" "V33"
[34] "V34" "V35" "V36" "V37" "V38" "V39" "V40" "V41" "V42" "V43" "V44"
[45] "V45" "V46" "V47" "V48" "V49" "V50" "V51" "V52" "V53" "V54" "V55"
[56] "V56" "V57" "V58" "V59" "V60" "V61" "V62" "V63" "V64"
```

```
View(dataset)
```

I dati sono strutturati in una matrice di dimensioni 64×6830 dove le righe rappresentano le linee cellulari tumorali e le colonne rappresentano i geni. I valori indicano i livelli di espressione genica.

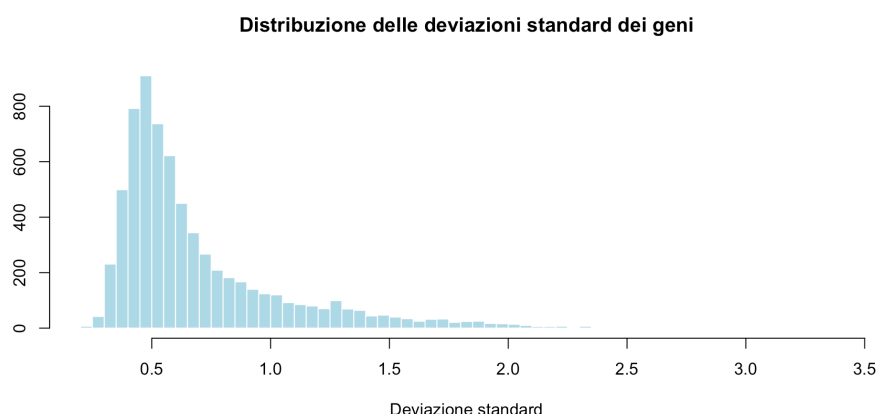
```
medie_geni = apply(dataset, 2, mean)
sd_geni = apply(dataset, 2, sd)
var_geni = apply(dataset, 2, var)

summary(medie_geni)
      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
-0.862188 -0.033592  0.009687  0.020148  0.054816  1.167646

summary(sd_geni)
      Min. 1st Qu.  Median     Mean 3rd Qu.     Max.
 0.2014  0.4571  0.5651  0.6964  0.8029  3.4238
```

Le medie dei geni sono vicine a zero, il che indica che i dati potrebbero essere già stati centrati. La scala dei valori è abbastanza ristretta, con la maggior parte dei geni tra -0.86 e 1.17. Il range di deviazione standard è abbastanza ampio (da 0.20 a 3.42), indicando che alcuni geni variano molto mentre altri sono quasi costanti.

```
hist(sd_geni, breaks = 50, col = "lightblue", main = "Distribuzione delle deviazioni standard
dei geni", xlab = "Deviazione standard", ylab = "", border = "white")
```



La distribuzione delle deviazioni standard dei geni presenta una coda lunga destra, con la maggior parte dei geni che presenta una variazione minima tra le linee cellulari (tra 0.2 e 1.0), con un picco intorno a 0.5, suggerendo che potrebbero essere meno informativi. Tuttavia, alcuni geni mostrano una deviazione standard elevata, fino a 3.5, indicando un'espressione molto variabile tra le cellule.

2. Applicazione della PCA

L'istogramma delle deviazioni standard ha mostrato che la maggior parte dei geni ha una bassa variabilità, mentre pochi geni presentano una varianza elevata. Decido quindi di procedere con una standardizzazione per impedire ai geni con un'alta varianza di dominare le componenti principali, cercando una rappresentazione più equilibrata delle informazioni.

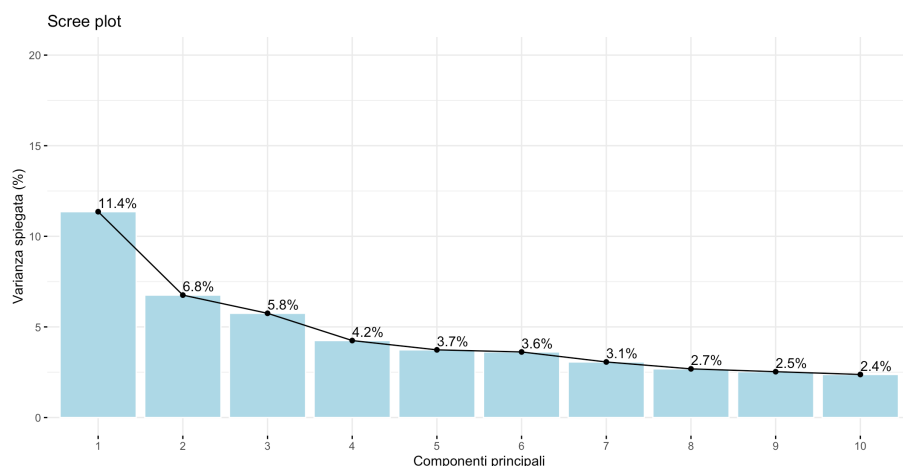
```
pca = PCA(dataset, scale.unit = TRUE, graph = FALSE)

pca$eig
```

	eigenvalue	percentage of variance
comp 1	775.81573	11.3589419
comp 2	461.44863	6.7562025
comp 3	392.85082	5.7518422
comp 4	290.10797	4.2475545
comp 5	255.09861	3.7349724
comp 6	247.15244	3.6186302
comp 7	209.42299	3.0662224
comp 8	183.44718	2.6859031
comp 9	172.76472	2.5294981
comp 10	162.27184	2.3758688

L'analisi degli autovalori della PCA mostra che la prima componente principale (PC1) spiega l'11.36% della varianza, seguita da PC2 con il 6.76%, mentre le prime due componenti insieme spiegano solo il 18.12% della varianza totale. La varianza spiegata diminuisce rapidamente, indicando che il dataset è altamente disperso e non facilmente riducibile a poche dimensioni. Poiché le prime 10 componenti spiegano solo circa il 46% della varianza totale, una visualizzazione bidimensionale potrebbe non essere sufficiente per catturare tutta la struttura dei dati.

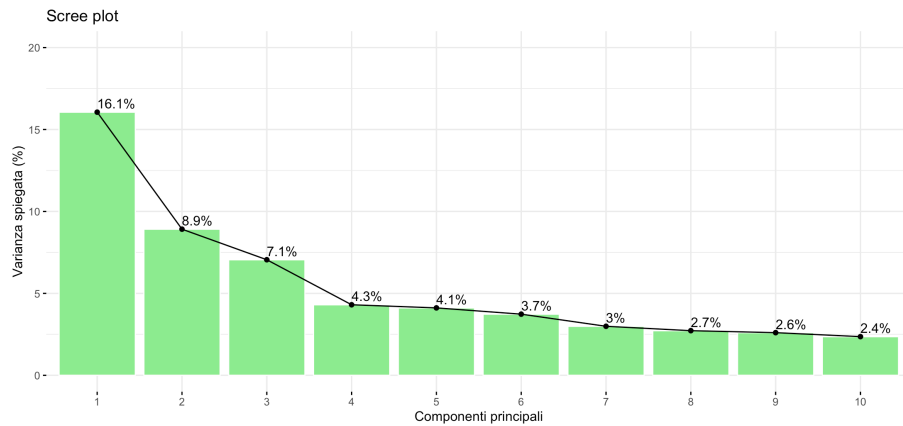
```
fviz_screplot(pca,
  addlabels = TRUE,
  ylim = c(0, 20),
  main = "Scree plot",
  xlab = "Componenti principali",
  ylab = "Varianza spiegata (%)",
  barfill = "lightblue",
  barcolor = "white")
```



Selezioniamo i geni in base alla deviazione standard, per mantenere solo le variabili più informative ed eliminare quelle con variazione minima tra le linee cellulari tumorali. Consideriamo come soglia il 75° percentile della distribuzione delle deviazioni standard; questo permette di mantenere circa il 25% dei geni più variabili, riducendo il rumore e migliorando l'efficacia della PCA.

```
soglia_sd = quantile(sd_geni, 0.75)
geni_selezionati = NCI60$data[, sd_geni > soglia_sd]
dim(geni_selezionati)
[1] 64 1708
```

```
pca_filtrata = PCA(geni_selezionati, scale.unit = TRUE, graph = FALSE)
fviz_screepplot(pca_filtrata,
  addlabels = TRUE,
  ylim = c(0, 20),
  main = "Scree plot",
  xlab = "Componenti principali",
  ylab = "Varianza spiegata (%)",
  barfill = "lightgreen",
  barcolor = "white")
```



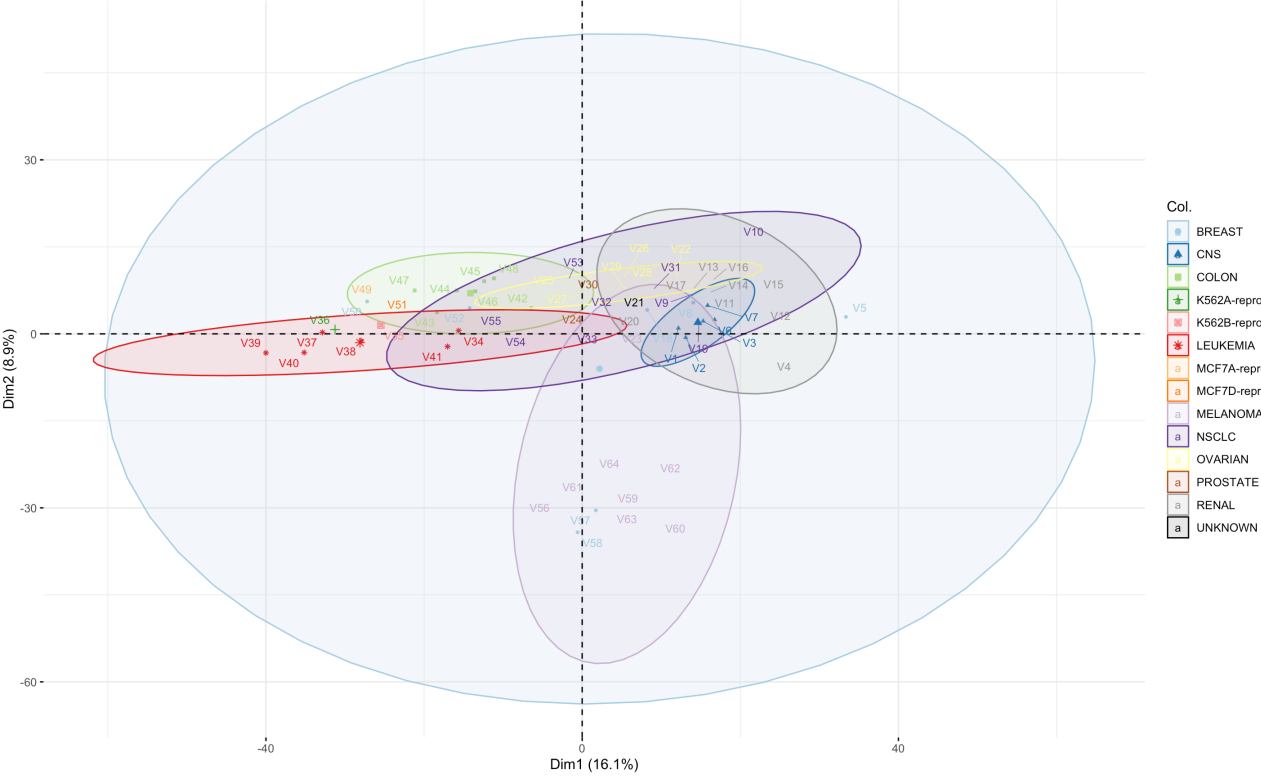
Dal grafico emerge che PC1 e PC2 spiegano insieme circa il 25% della varianza totale (16.1% e 8.9% rispettivamente). Le componenti successive, come PC3 (7.1%) e PC4 (4.3%), continuano a contribuire alla spiegazione della varianza, mentre a partire da PC5 l'incremento di varianza spiegata diventa progressivamente minore. In questo caso, il calo brusco dopo PC3 o PC4 suggerisce che mantenere 3-5 componenti principali potrebbe essere una scelta equilibrata per preservare l'informazione senza introdurre troppa complessità. Dopo PC5, la varianza spiegata da ciascuna componente si riduce significativamente, indicando che quelle componenti contengono informazioni meno rilevanti.

```
palette_colori <- brewer.pal(n = 12, name = "Paired")
palette_colori <- c(palette_colori, "#999999", "#000000")

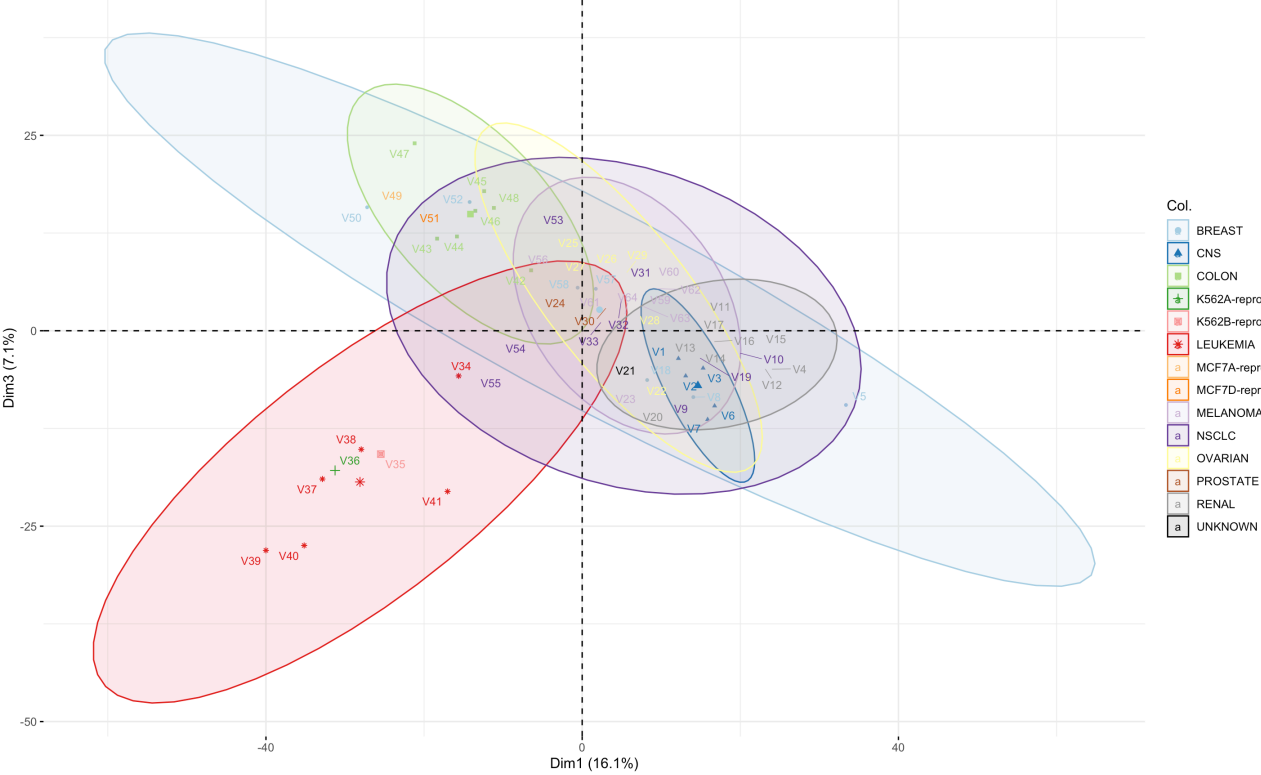
fviz_pca_ind(pca_filtered,
  col.ind = NCI60$labs,
  palette = palette_colori,
  addEllipses = TRUE,
  repel = TRUE,
  labelsize = 3,
  pointsize = 1)

fviz_pca_ind(pca_filtered,
  axes = c(1,3),
  title = "PCA (PC1 vs PC3)",
  col.ind = NCI60$labs,
  palette = palette_colori,
  addEllipses = TRUE,
  repel = TRUE,
  labelsize = 3,
  pointsize = 1)
```

PCA (PC1 vs PC2)



PCA (PC1 vs PC3)



Il grafico PCA rappresenta la distribuzione delle linee cellulari tumorali lungo diverse componenti principali, con PC1 (16.1%) combinata prima con PC2 (8.9%) e poi con PC3 (7.1%), spiegando rispettivamente il 25% e il 23.2% della varianza totale.

Le linee cellulari sono colorate in base al tipo di tumore, con le ellissi che indicano la dispersione dei gruppi e quindi la loro eterogeneità interna. Alcuni tumori mostrano una chiara separazione, come la leucemia (rosso), ben distinta dagli altri e più evidente nella PCA con PC3, suggerendo che questa componente cattura caratteristiche genetiche specifiche di questo tumore. Il NSCLC (viola) risulta molto disperso, indicando un'ampia variabilità intra-tumorale. Al contrario, altri gruppi come mammella (azzurro), rene (grigio), colon (verde) e ovaio (giallo) mostrano sovrapposizioni significative, suggerendo possibili pathway biologici comuni o la necessità di componenti aggiuntive per una distinzione più netta.

La dimensione delle ellissi riflette l'eterogeneità interna di ciascun tumore: gruppi come breast e NSCLC hanno ellissi più grandi, indicando una maggiore variabilità nei profili di espressione, mentre gruppi più compatti presentano una maggiore omogeneità. Complessivamente, l'analisi PCA offre una buona separazione per alcuni tumori ma non per tutti, suggerendo che ulteriori componenti principali (PC4, PC5) potrebbero essere necessari per una classificazione più chiara.

```
contrib_totale <- rowSums(pca_result$var$contrib[, 1:5])
top_100_geni <- order(contrib_totale, decreasing = TRUE)[1:100]
```

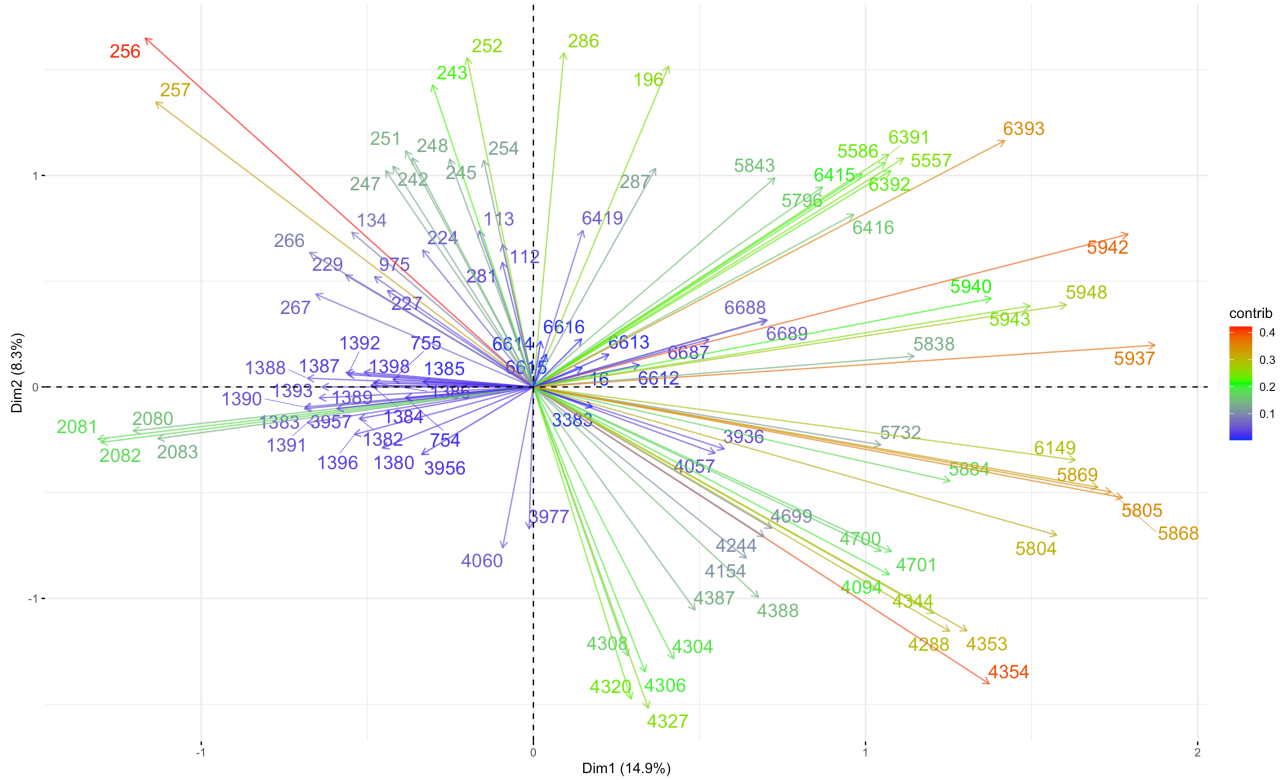
Per selezionare i geni più rilevanti nella PCA, calcoliamo il contributo totale di ciascun gene sulle prime cinque componenti principali (PC1 - PC5), ottenendo una misura complessiva dell'importanza di ogni variabile nell'analisi. Successivamente ordiniamo i geni in base al loro contributo totale, dal più influente al meno influente, per identificare quelli che hanno il maggiore impatto sulla separazione delle linee cellulari tumorali. A questo punto, selezioniamo i primi 100 geni e li utilizziamo per la visualizzazione. Infine, creiamo il grafico della PCA mostrando solo queste variabili selezionate, migliorando la leggibilità e permettendo di evidenziare i geni più determinanti per la variabilità nei dati.

```
fviz_pca_var(pca_result,
  select.var = list(name = rownames(pca_result$var$contrib)[top_100_geni]),
  col.var = "contrib",
  gradient.cols = c("blue", "green", "red"),
  alpha.var = 0.7,
  labelsize = 5,
  arrow.alpha = 0.6,
  repel = TRUE)

fviz_pca_var(pca_result,
  axes = c(1,3),
  title = "Contributo dei geni a PC1 e PC3",
  select.var = list(name = rownames(pca_result$var$contrib)[top_100_geni]),
  col.var = "contrib",
  gradient.cols = c("blue", "green", "red"),
  alpha.var = 0.7,
  labelsize = 5,
  arrow.alpha = 0.6,
  repel = TRUE)
```

I grafici rappresentano il contributo dei geni alle componenti principali della PCA, evidenziando quali influenzano maggiormente la separazione delle linee tumorali. Ogni freccia indica un gene, con la direzione che mostra la correlazione con le componenti principali e la lunghezza che riflette il grado di associazione. Il colore rappresenta il contributo con i geni più influenti evidenziati in rosso e giallo, mentre quelli meno rilevanti sono in blu. I geni con il contributo più alto, come 256 e 5937, si trovano nelle zone periferiche, suggerendo un ruolo chiave nella definizione della variabilità tra le linee tumorali. Altri geni, come 2081, 2082, 5805 e 5942, mostrano un contributo significativo, mentre quelli più vicini al centro hanno un impatto minore e potrebbero essere meno informativi per la classificazione. L'inclusione di PC3 modifica la distribuzione delle variabili, rivelando geni che potrebbero non essere stati evidenti nella PCA su PC1 e PC2, suggerendo che PC3 cattura informazioni complementari. Dal punto di vista biologico, i geni con alto contributo potrebbero essere biomarcatori chiave per la separazione dei tumori.

Contributo dei geni a PC1 e PC2



ESERCIZIO RELATIVO LEZ. 16-17 Bogdan Delia Anamaria

NB: il testo riporta la totalità delle lezioni 16-17, le parti da completare per esercizio sono indicate in rosso.

Analizziamo il dataset Golub (1999), che contiene l'espressione di 3051 geni in 38 pazienti affetti da due tipi di leucemia; ALL (Leucemia Linfoblastica Acuta) per 27 pazienti e AML (Leucemia Mieloide Acuta) per 11 pazienti.

```
library(multtest)
data(golub)
dim(golub)
[1] 3051 38

table(golub.cl)
golub.cl
0 1
27 11

golub_tr <- t(golub)
D.golub <- dist(x = golub_tr, method = "euclidean")
library(cluster)
D.golub.bis <- daisy(x = golub_tr, metric = "euclidean")
sum(abs(D.golub - D.golub.bis))
[1] 0
```

Dopo la trasposizione, le righe rappresentano i 38 pazienti e le colonne i 3051 geni, permettendo di calcolare le distanze tra i campioni in base ai loro profili di espressione genica. Calcoliamo la matrice di distanze euclidee con funzioni provenienti da pacchetti diversi: il risultato è una matrice di distanza 38 x 38 (simmetrica), dove ogni cella rappresenta la distanza tra due pazienti. La distanza euclidea misura la similarità tra ogni coppia di pazienti, quantificando le differenze nei livelli di espressione genica.

```
results.agnes <- agnes(x = golub_tr, diss = FALSE)
results.agnes.3 <- agnes(x = D.golub, diss = TRUE)

Call: agnes(x = golub_tr, diss = FALSE)
Agglomerative coefficient: 0.3494558
Order of objects:
[1] 1 4 7 8 22 27 5 15 24 13 16 19 26 20 18 2 14 3 6 23 9 11 10
[24] 17 12 25 28 31 32 34 35 30 36 29 33 37 38 21
Height (summary):
  Min. 1st Qu. Median Mean 3rd Qu. Max.
 27.76 33.79 37.59 37.67 40.85 54.29
```

agnes() utilizza di default il legame medio per unire i cluster. results.agnes e results.agnes.3 restituiscono lo stesso risultato perché sono stati eseguiti con le stesse impostazioni, uno con i dati originali e uno con la matrice di distanza pre-calcolata.

```
t(results.agnes$merge[1:25,])

      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12]
[1,]   -5   -6   -4   -9   -3    1  -16    7    6   -30   -34    4
[2,]  -15  -23   -7  -11    2  -24  -19  -26  -13   -36   -35  -10

      [,13] [,14] [,15] [,16] [,17] [,18] [,19] [,20] [,21] [,22] [,23] [,24] [,25]
[1,]     5   -33   -31   -12   -1  -28    9   -2   18   -8   20   19   17
[2,]    12  -37  -32  -25    3   15    8  -14   11  -22   13  -20   22
```

merge è una matrice (n-1) x 2 che mostra come i cluster si uniscono in ogni step. Valori negativi indicano un singolo elemento, valori positivi indicano cluster già esistenti.

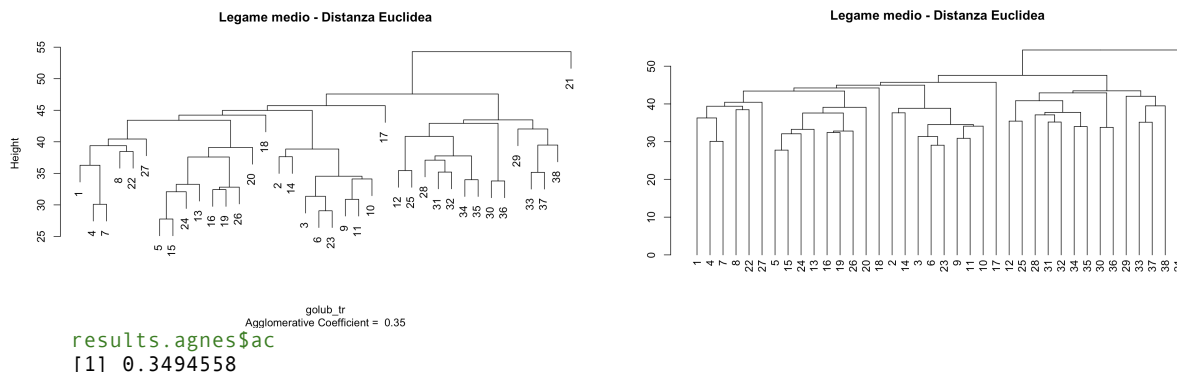
```
> results.agnes$height
[1] 36.28464 30.09425 39.38917 38.47159 40.44182 43.41314 27.76315 32.08506 33.26133
[10] 37.59122 32.41944 32.81579 39.10573 44.23222 44.97620 37.63986 38.85445 31.36104
[19] 29.06565 34.54523 30.88571 34.11213 45.73110 47.57777 35.44435 40.85357 37.09546
[28] 35.20125 37.76506 33.98912 42.92614 33.79348 43.47253 42.03364 35.16133 39.48551
[37] 54.29456
```

height mostra a quale distanza due cluster sono stati uniti. Maggiore è il valore, più distanti erano i cluster prima della fusione.

```
> results.agnes$order
[1] 1 4 7 8 22 27 5 15 24 13 16 19 26 20 18 2 14 3 6 23 9 11 10 17 12 25 28 31
[29] 32 34 35 30 36 29 33 37 38 21
```

order determina l'ordine in cui i pazienti vengono disposti nel dendrogramma.

```
plot(results.agnes, which.plots = 2, main = "Legame medio - Distanza Euclidea")
plot(as.dendrogram(results.agnes), main = "Legame medio - Distanza Euclidea")
```



Il coefficiente agglomerativo (ac) misura quanto bene la struttura gerarchica cattura la separazione tra i cluster. Valori bassi come questo (0.35) indicano che i cluster non sono ben definiti.

```
res.hclust <- hclust(d = D.golub)
res.hclust$merge; res.hclust$height; res.hclust$order
```

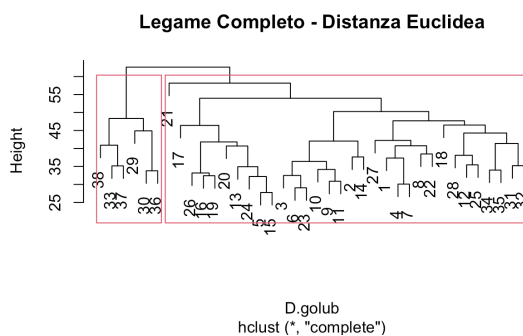
hclust crea un clustering gerarchico basato sulla matrice di distanza D.golub. Il metodo di default è complete linkage, che calcola la distanza massima tra elementi di cluster diversi.

```
cluster.hclust <- cutree(res.hclust, k = 2)
cluster.hclust.2 <- cutree(res.hclust, h = 59)
table(cluster.hclust; table(cluster.hclust.2))
```

```
cluster.hclust
 1  2
32  6
```

cutree divide i dati in 2 cluster (un cluster con 32 unità e l'altro con 6 unità), tagliando il dendrogramma a un livello appropriato. In alternativa, possiamo tagliare il dendrogramma a un'altezza specifica (h = 59).

```
plot(res.hclust, main = "Legame Completo - Distanza Euclidea")
rect.hclust(res.hclust, k = 2)
```



```
table(cluster.hclust, golub.cl)

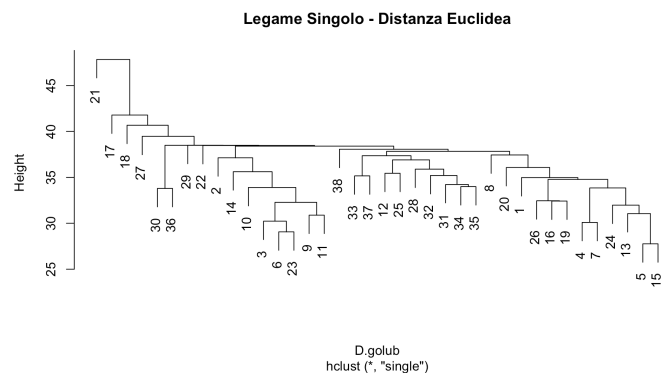
      golub.cl
cluster.hclust 0  1
               1 27 5
               2  0 6

adjustedRandIndex(cluster.hclust, golub.cl)
[1] 0.495057
```

Il confronto tra il clustering ottenuto (cluster.hclust) e le classi note (golub.cl) mostra che tutti i 27 pazienti ALL (0) sono stati assegnati correttamente al cluster 1 e 6 pazienti AML (1) sono stati correttamente assegnati al cluster 2. Tuttavia, 5 pazienti AML sono stati erroneamente classificati nel cluster 1. L'Adjusted Rand Index (ARI) di 0.495 indica una moderata concordanza tra il clustering e le classi note. In questo caso, l'ARI non è altissimo, suggerendo che il clustering cattura alcune informazioni sulla struttura dei dati, ma potrebbe non essere perfettamente allineato con la classificazione ALL/AML.

```
res.hclust.single <- hclust(d = D.golub, method = "single")
plot(res.hclust.single, main = "Legame Singolo - Distanza Euclidea")
```

Il Single Linkage unisce i cluster in base alla distanza minima tra i punti.



```
res.hclust.ward <- hclust(d = D.golub, method = "ward.D")
res.hclust.centroid <- hclust(d = D.golub, method = "centroid")
```

Implementiamo il metodo di Ward e del centroide. Il metodo del centroide calcola la distanza tra i cluster basandosi sulla distanza tra i centri medi dei gruppi. È meno sensibile agli outlier, ma può generare inversioni nell'ordine della fusione; il metodo di Ward è un algoritmo di clustering gerarchico che minimizza la varianza all'interno dei cluster, garantendo gruppi più compatti rispetto ad altri metodi.

```
cluster.ward <- cutree(res.hclust.ward, k = 2)
cluster.centroid <- cutree(res.hclust.centroid, k = 2)
```

Dividiamo i dati in 2 cluster in base alle strutture dei dendrogrammi.

```
table(cluster.ward, golub.cl)
      golub.cl
cluster.ward 0  1
               1 25 0
               2  2 11

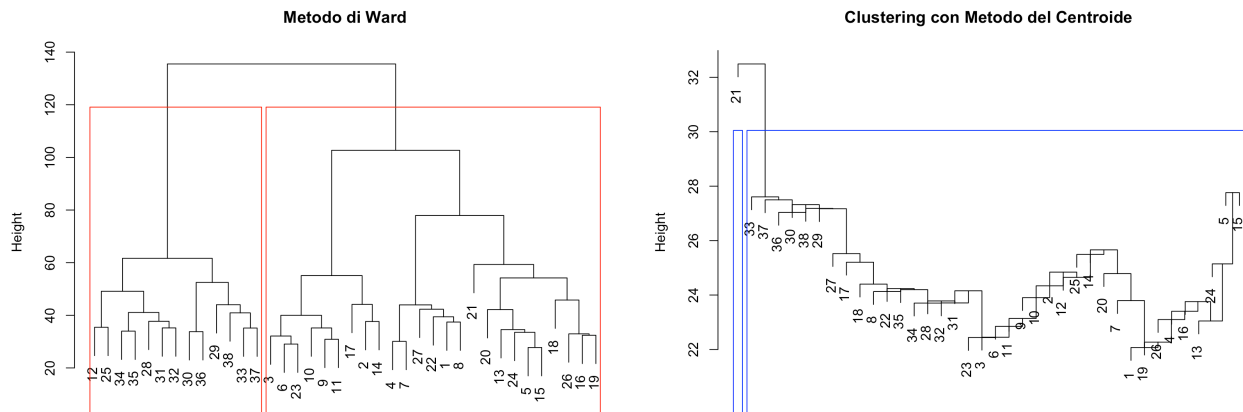
table(cluster.centroid, golub.cl)
      golub.cl
cluster.centroid 0  1
                  1 26 11
                  2  1 0
```

Il metodo di Ward ha prodotto una separazione più coerente tra ALL e AML, con solo due errori, mentre il metodo del centroide non ha distinto bene le due classi, creando un cluster con una mescolanza di entrambi i tipi di leucemia. Ward sembra quindi più efficace per questo dataset.

```
adjustedRandIndex(cluster.ward, golub.cl)
[1] 0.7927409
adjustedRandIndex(cluster.centroid, golub.cl)
[1] -0.03059805
```

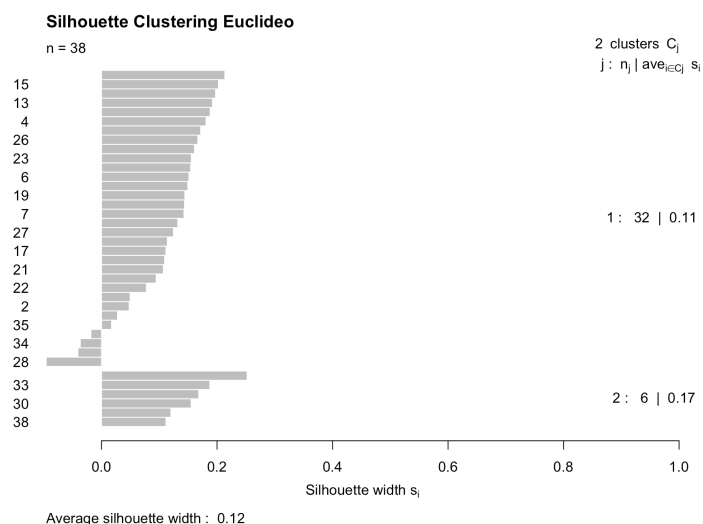
Il valore di 0.79 indica che il clustering di Ward è ben allineato con la classificazione effettiva. Il metodo del Centroide restituisce un valore negativo che indica che il clustering è peggiore della casualità, ossia i pazienti sono stati assegnati ai cluster in modo inefficace.

```
plot(res.hclust.ward, main = "Metodo di Ward", sub = "", xlab = "")
rect.hclust(res.hclust.ward, k = 2, border = "red")
plot(res.hclust.centroid, main = "Clustering con Metodo del Centroide", sub = "", xlab = "")
rect.hclust(res.hclust.centroid, k = 2, border = "blue")
```



Il dendrogramma ottenuto con il metodo di Ward (sinistra) mostra cluster ben distinti e bilanciati, con un incremento graduale delle altezze di fusione, suggerendo una gerarchia stabile. I rettangoli rossi evidenziano i cluster ben separati, confermando che Ward minimizza la varianza intra-cluster e fornisce una struttura gerarchica chiara e interpretabile. Al contrario, il metodo del centroide (destra) appare meno stabile, con alcune fusioni che sembrano innaturali e inversioni nell'ordine di fusione, in cui i rami scendono invece di salire. Questo indica instabilità nella struttura dei cluster, rendendo la segmentazione meno chiara rispetto a Ward e confermando che il metodo del centroide può essere meno affidabile in alcune situazioni.

```
silhouette_euclidean <- silhouette(cutree(res.hclust, k = 2), D.golub)
plot(silhouette_euclidean, main = "Silhouette Clustering Euclidean")
```



Il clustering basato sulla distanza euclidea non è ben definito, con molte osservazioni che presentano silhouette basse o vicine a zero, suggerendo la necessità di aumentare il numero di cluster ($k > 2$) o di testare un metodo di clustering diverso.

```

k <- 2

mean.cluster <- t(sapply(X = 1:k, FUN = function(nc)apply(golub_tr[cluster.hclust.ward == nc,
1:ncol(golub_tr)], 2, mean)))

dim(mean.cluster)
[1] 2 3051

round(mean.cluster[1,], 2)
[1] -1.28 -0.92 0.29 0.85 0.56 0.59 2.97 2.87 3.05 3.08 -0.56 -1.30 -0.64 0.86
[15] 1.56 0.41 1.22 -0.78 -0.33 -0.26 -0.69 -0.74 -0.51 0.53 0.05 -1.32 -0.19 0.70
...

round(mean.cluster[2,], 2)
[1] -0.84 -0.72 0.21 1.13 0.30 0.48 3.23 3.13 3.09 3.21 0.10 -0.57 0.28 1.12
[15] 1.84 0.50 0.99 -0.26 -0.03 0.07 -0.28 -0.41 -1.02 0.44 -0.68 -0.93 -0.30 0.71
...

```

Abbiamo diviso i geni in 2 cluster utilizzando un algoritmo gerarchico di Ward. Per ogni cluster, si calcola la media di ogni colonna (gene). Queste medie vengono memorizzate in una matrice di dimensione 2×3051 , dove ciascuna delle due righe corrisponde alle medie di un cluster su tutte le colonne.

```

diff_genes <- mean.cluster[1,] - mean.cluster[2,]
top_genes_index <- order(abs(diff_genes), decreasing = TRUE)
head(top_genes_index)
[1] 2664 2663 829 773 1009 904

```

Definiamo la differenza per ogni gene: se per un gene la differenza è positiva, significa che il cluster 1 ha una media maggiore per quel gene. Per individuare i geni che contribuiscono maggiormente alla separazione, li ordiniamo in base al valore assoluto della differenza. Indichiamo nell'output le posizioni dei primi 6 geni discriminanti, ordinati in base al valore assoluto della differenza tra le medie dei due cluster.

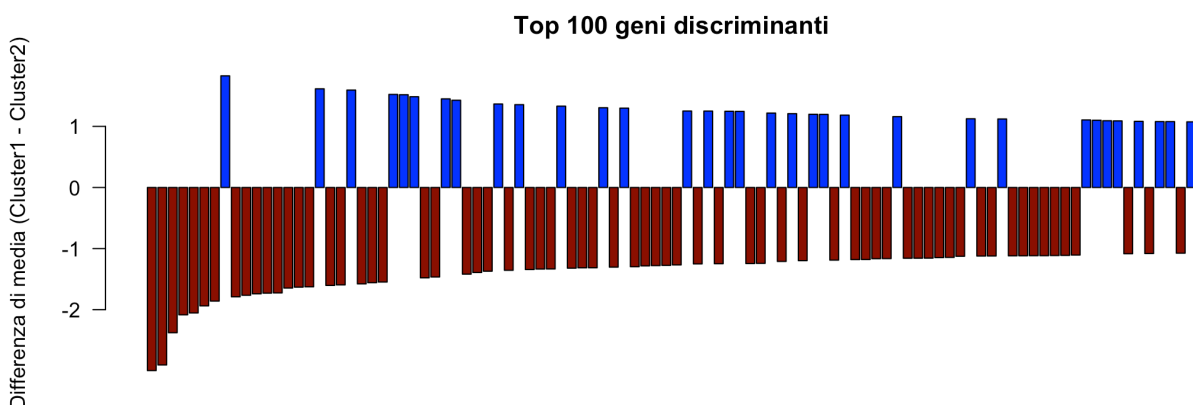
```

ordered_index <- order(abs(top100_diff), decreasing = TRUE)
top100_diff_sorted <- top100_diff[ordered_index]

barplot(top100_diff_sorted,
        main = "Top 100 geni discriminanti",
        col = ifelse(top100_diff_sorted > 0, "blue", "darkred"),
        las = 2,
        cex.names = 0.7,
        ylab = "Differenza di media (Cluster1 - Cluster2)",
        border = "black")

abline(h = 0, col = "black", lwd = 1.5, lty = 2)

```



Il grafico mostra le differenze di espressione media tra Cluster1 e Cluster2 per i 100 geni più discriminanti, ovvero quelli con la maggiore differenza assoluta nei livelli di espressione tra i due gruppi. Sull'asse Y è riportata la differenza di espressione media (Cluster1 - Cluster2), dove valori positivi (barre blu) indicano geni più espressi in Cluster1 e valori negativi (barre rosse) indicano geni più espressi in Cluster2. Sull'asse X sono rappresentati i 100 geni più discriminanti, ordinati in base alla differenza assoluta di espressione tra i due cluster, con quelli più marcati a sinistra e quelli meno marcati a destra.

Dall'analisi del grafico emerge che la maggior parte dei geni più discriminanti presentano una maggiore espressione in Cluster2 (barre rosse), mentre alcuni geni, in particolare nella parte destra del grafico, mostrano una maggiore espressione in Cluster1 (barre blu).

```
res_kl <- NbClust(data = golub_tr, distance = "euclidean", method = "ward.D2", min.nc = 2,
max.nc = 20, index = "kl")
res_ch <- NbClust(data = golub_tr, distance = "euclidean", method = "ward.D2", min.nc = 2,
max.nc = 20, index = "ch")
res_hartigan <- NbClust(data = golub_tr, distance = "euclidean", method = "ward.D2", min.nc =
2, max.nc = 20, index = "hartigan")

res_kl$Best.nc
Number_clusters Value_Index
2.0000 1.5162
res_ch$Best.nc
Number_clusters Value_Index
2.000 5.508
res_hartigan$Best.nc
Number_clusters Value_Index
3.0000 0.9675
```

Utilizziamo la funzione `NbClust()` per determinare il numero ottimale di cluster nel dataset `golub_tr` utilizzando diversi indici di validità. Il metodo di clustering scelto è quello di Ward con distanza euclidea, e il numero di cluster viene cercato nell'intervallo tra 2 e 20. Vengono calcolati tre diversi indici: indice di Krzanowski e Lai (KL), indice di Calinski-Harabasz (CH), indice di Hartigan. Due indici (KL e CH) suggeriscono che 2 è il numero ottimale di cluster, un indice (Hartigan) suggerisce invece 3 cluster.

```
indices <- c("kl", "ch", "hartigan", "silhouette", "gap")
results <- data.frame(Index = indices, Best_k = NA)

for (i in seq_along(indices)) {
  res <- NbClust(data = golub_tr, distance = "euclidean", method = "ward.D2",
min.nc = 2, max.nc = 20, index = indices[i])
  results$Best_k[i] <- res$Best.nc
}

print(results)
```

	Index	Best_k
1	kl	2
2	ch	2
3	hartigan	3
4	silhouette	19
5	gap	2

Il codice esegue il calcolo del numero ottimale di cluster utilizzando diversi indici di validità disponibili nel pacchetto `NbClust`. Per prima cosa, viene creata una lista `indices` contenente i nomi di sette indici di validità: Krzanowski-Lai (KL), Calinski-Harabasz (CH), Hartigan, Silhouette e Gap Statistic. Successivamente, viene inizializzata una tabella (`results`) con due colonne: una per il nome dell'indice e una (`Best_k`) che sarà riempita con il numero ottimale di cluster individuato da ciascun indice.

Il codice utilizza un ciclo `for` per iterare su tutti gli indici e applicare la funzione `NbClust()` al dataset `golub_tr`, utilizzando la distanza euclidea e il metodo di Ward (`ward.D2`). Il numero di cluster ottimale viene cercato nell'intervallo da 2 a 20, e per ogni indice viene estratto il valore suggerito (`Best.nc`) e memorizzato nella tabella `results`.

La regola della maggioranza suggerisce che il miglior numero di cluster è 2.

```

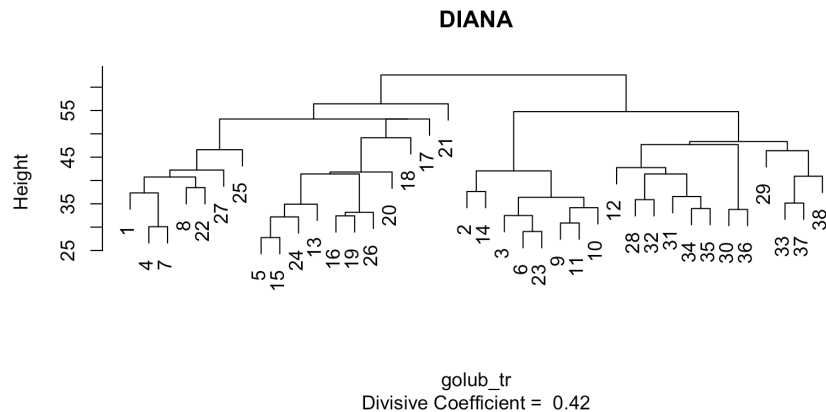
res.diana <- diana(x = golub_tr, diss = FALSE)
plot(res.diana, main = "DIANA", which.plots = 2)
cluster.diana <- cutree(res.diana, k = 2)
table(cluster.diana)
table(cluster.diana, golub.cl)

      golub.cl
cluster.diana  0  1
              1 18  0
              2  9 11

adjustedRandIndex(cluster.diana, golub.cl)

chisq.test(cluster.diana, golub.cl)$p.value
[1] 0.0007393755

```



Il metodo DIANA viene applicato per eseguire un clustering divisivo sui dati. Il clustering ottenuto viene confrontato con una classificazione nota, per cui risulta che il Cluster 1 contiene esclusivamente 18 elementi della classe 0, mentre il Cluster 2 è più misto, includendo 9 elementi della classe 0 e 11 della classe 1. Questo suggerisce che il clustering è parzialmente coerente con la classificazione nota: il Cluster 1 separa bene la classe 0, mentre il Cluster 2 presenta una sovrapposizione tra le due classi. La significatività della partizione viene verificata attraverso il test chi-quadrato, il cui p-value risulta essere di 0.0007393755, un valore molto basso, il che suggerisce che esiste una dipendenza significativa tra il clustering e la classificazione originale.

L'obiettivo è determinare il numero ottimale di cluster k utilizzando il metodo del gomito osservando l'andamento della somma delle varianze within.

```

n <- dim(golub_tr)[1]
k.min <- 2; k.max <- 20

wss <- numeric()
wss[1] <- (n - 1)*sum(apply(golub_tr, 2, var)) # totale
for(k in k.min:k.max){
  wss[k] <- kmeans(golub_tr, centers = k, nstart = 50)$tot.withinss}
wss

[1] 38558.13 33441.53 29977.02 27539.09 25870.63 24206.46 22875.14 21563.66 20268.34
[10] 19099.66 18079.20 17114.90 16145.65 15188.63 14312.55 13432.09 12561.97 11727.26
[19] 10950.66 10183.96

diff(wss)

[1] -5116.5983 -3464.5059 -2437.9335 -1668.4628 -1664.1713 -1331.3183 -1295.3194
[8] -1311.4796 -1168.6802 -1020.4574 -969.2469 -943.7169 -977.6134 -876.0754
[15] -868.0955 -882.4844 -776.6077 -834.7056 -766.6958

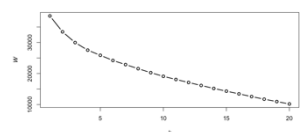
```

Il vettore wss mostra che la varianza interna ai cluster diminuisce all'aumentare del numero di cluster, confermando che K-means sta migliorando la separazione dei gruppi.

```

plot(1:k.max, wss,
     xlab = expression(italic(k)),
     ylab = expression(italic(W)),
     type = "b", lwd = 2)

```



Il metodo del gomito mostra che oltre un certo k, la riduzione di W non è più rilevante. k=3 o 4 sembrano essere buoni valori per il numero di cluster.

```

wss <- numeric()
wss[1] <- (n - 1)*sum(apply(golub_tr, 2, var))

for(k in k.min:k.max){
  wss[k] <- kmeans(golub_tr, k, nstart=50, iter.max=1000)$tot.withinss
}

plot(1:k.max, wss,      xlab = expression(italic(k)),
     ylab = expression(italic(W)),
     type = "b", lwd = 2)

diff(wss)

[1] -5116.5983 -3464.5059 -2437.9335 -1668.4628 -1664.1713 -1331.3183 -1295.3194
[8] -1311.4796 -1168.6802 -1020.4574 -969.2469 -943.7169 -977.6134 -876.0754
[15] -868.0955 -882.4844 -776.6077 -834.7056 -766.6958

```

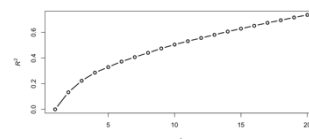
Cambiando starting point ritroviamo la stessa soluzione, ciò vuol dire che eravamo già giunti all'ottimo locale.

```

R2 <- 1 - wss/wss[1]
[1] 0.0000000 0.1326983 0.2225498 0.2857773 0.3290487 0.3722087 0.4067363 0.4403302
[9] 0.4743433 0.5046528 0.5311183 0.5562556 0.5807307 0.6060850 0.6288059 0.6513198
[17] 0.6742070 0.6943482 0.7159962 0.7358803

plot(1:k.max, R2,
     xlab = expression(italic(k)),
     ylab = expression(italic(R^2)),
     type = "b", lwd = 2)

```

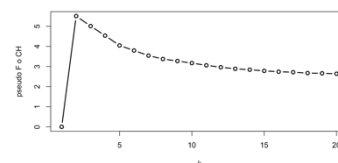


R2 è un altro indice per valutare la qualità del clustering: più è alto, più i cluster sono distinti.

```

bss <- wss[1] - wss
pseudoF <- numeric()
pseudoF[1] <- 0
for(k in k.min:k.max){
  pseudoF[k] <- (bss[k]/(k - 1))/(wss[k]/(n - k))
}
plot(1:k.max, pseudoF,
     xlab = expression(italic(k)),
     ylab = "pseudo F o CH",
     type = "b", lwd = 2,)

```



Il numero ottimale di cluster è quello in cui il valore di Pseudo-F è massimo, ovvero in questo caso 2.

```

res.kmeans.3 <- kmeans(golub_tr,
                      centers = 2,
                      nstart = 50,
                      iter.max = 1000)

res.kmeans.3$totss
[1] 38558.13

res.kmeans.3$betweenss
[1] 5116.598

res.kmeans.3$tot.withinss
[1] 33441.53

res.kmeans.3$withinss
[1] 10525.76 22915.77

res.kmeans.3$size
[1] 13 25

res.kmeans.3$iter
[1] 1
res.kmeans.3$ifault
[1] 0
res.kmeans.3$cluster
[1] 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 1 2 2 1 1 1 1 1 1 1 1 1 1 1

table(res.kmeans.3$cluster)
table(res.kmeans.3$cluster, golub.cl)
  golub.cl
    0     1
  1  2  11
  2 25   0

```

```

res.kmeans.3$centers <- t(apply(X = 1:k,
                                FUN = function(nc)apply(
                                    golub_tr[res.kmeans.3$cluster == nc, ],
                                    2, mean)))
rownames(res.kmeans.3$centers) <- 1:k
round(res.kmeans.3$centers, 2)

```

Calcolo dei valori medi (sovrascritti sui centroidi di k-means) delle variabili originali per ciascun cluster.

[illegible]

Il valore della funzione obiettivo è costantemente uguale a 83.8061 per tutti i 20 tentativi. Questo indica che l'algoritmo PAM converge sempre allo stesso minimo locale indipendentemente dall'inizializzazione. Dall'output vediamo che il cluster più grande è il primo con 23 unità, mentre il secondo ne contiene 15. "16" e "28" sono invece gli indici dei medoidi, ovvero degli elementi del dataset che rappresentano i centri nel cluster in PAM. Per valutare la separazione dei cluster, analizziamo il diametro e la separazione. Nel nostro caso, il Cluster 1 ha un diameter di 148.3399 e una separation di 82.35422, mentre il Cluster 2 ha un diameter di 116.3509 con la stessa separation di 82.35422. Poiché la separation è inferiore ai diametri di entrambi i cluster, ciò suggerisce una certa sovrapposizione tra i gruppi.

```
table(res.pam.3$clustering, res.kmeans.3$cluster)
adjustedRandIndex(res.pam.3$clustering, res.kmeans.3$cluster)
chisq.test(res.pam.3$clustering, res.kmeans.3$cluster)$p.value
[1] 2.541144e-07
```

Un p-value molto basso come questo indica che la distribuzione dei cluster ottenuti dai due metodi non è indipendente, ovvero che le soluzioni fornite da PAM e K-means sono fra loro correlate e simili.

```
res.pam.3$silinfo$avg.width
[1] 0.07236134

res.pam.3$silinfo$clus.avg.widths
[1] 0.06460863 0.08424881

res.pam.3$silinfo$widths[1:3, ]
  cluster neighbor sil_width
15       1         2 0.1328695
24       1         2 0.1243265
13       1         2 0.1216425
```

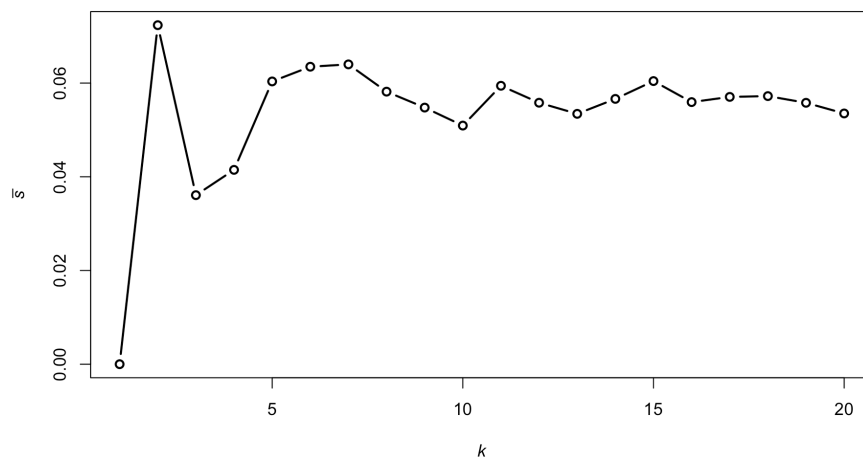
```

k.min <- 2
k.max <- 20
asw <- numeric(k.max)
asw[1] <- 0

for (k in k.min:k.max) {
  asw[k] <- pam(x = golub_tr, k = k, stand = TRUE)$silinfo$avg.width
}

plot(1:k.max, asw,
     xlab = expression(italic(k)),
     ylab = expression(bar(italic(s))),
     type = "b", lwd = 2)

```



La figura mostra l'andamento della silhouette media in funzione del numero di cluster k utilizzando l'algoritmo PAM. Un valore della silhouette vicino a 1 indica cluster ben separati, mentre valori prossimi a 0 suggeriscono elementi al confine tra cluster. Valori negativi indicano assegnazioni errate. Dall'analisi del grafico, si osserva che la silhouette media è relativamente alta per $k=2$ e, per valori maggiori, oscilla senza un incremento significativo. Questo suggerisce che due cluster potrebbero essere una scelta adeguata. L'assenza di un aumento netto della silhouette per $k>2$ indica che un numero maggiore di cluster non migliora significativamente la separazione tra i gruppi. Di conseguenza, il valore ottimale di k è quello che massimizza la silhouette media, e in questo caso sembra essere $k=2$.

```

run_cluster_simulations <- function(data, p_value, min_clust, max_clust) {
  distances <- paste0("d", 1:5)
  methods <- paste0("m", 1:5)

  results_list <- list()

  for (dist in distances) {
    for (meth in methods) {
      cat("Eseguendo cluster.Sim con distanza:", dist, "e metodo:", meth, "\n")

      result <- cluster.Sim(
        x = data,
        p = p_value,
        minClusterNo = min_clust,
        maxClusterNo = max_clust,
        icq = "KL",
        distances = c(dist),
        methods = c(meth) )

      results_list[[paste0("dist_", dist, "_meth_", meth)]] <- result
    }
  }

  return(results_list)}

```

```
ch_results <- run_cluster_simulations(golub_tr, p_value = 2, min_clust = 2, max_clust = 5)
```

Calinski-Harabasz	Single	Complete	Average	mcquitty	pam
Manhattan	2	2	3	4	2
Euclidean	2	2	3	2	2
Chebyshev	2	2	2	2	2
Squared Euclidean	2	3	3	2	2
GDM1	4	2	2	2	2

Krzanowski & Lai	Single	Complete	Average	mcquitty	pam
Manhattan	2	4	3	4	2
Euclidean	2	5	3	5	4
Chebyshev	2	3	2	5	5
Squared Euclidean	2	5	3	5	4
GDM1	4	2	3	4	4

Le due tabelle confrontano il numero di cluster selezionati secondo i criteri di validità Calinski-Harabasz (prima tabella) e Krzanowski & Lai (seconda tabella), analizzando diverse combinazioni di metriche di distanza (Manhattan, Euclidean, Chebyshev, Squared Euclidean, GDM1) e metodi di clustering (Single, Complete, Average, McQuitty, PAM). Il criterio Calinski-Harabasz tende a identificare meno cluster, con valori generalmente compresi tra 2 e 4, mentre Krzanowski & Lai mostra maggiore variabilità, con alcune combinazioni (es. Euclidean con Complete o McQuitty) che portano fino a 5 cluster. La distanza GDM1 con il metodo Single linkage genera sempre 4 cluster in entrambi i criteri, mentre altre distanze mostrano più variazione. Il metodo PAM fornisce risultati più stabili, con un numero di cluster che varia tra 2 e 5. In generale, il criterio Krzanowski & Lai favorisce una maggiore segmentazione rispetto a Calinski-Harabasz, che invece tende a produrre soluzioni più compatte. Questa differenza aiuta a comprendere quale criterio sia più adatto in base al livello di granularità desiderato nell'analisi dei cluster.

ESERCIZIO SULLE MISTURE **Bogdan Delia Anamaria**

```
library(multtest)
library(mclust)

data(golub)
dim(golub)
table(golub.cl)
golub_tr <- t(golub)
dim(golub_tr)
```

Analizziamo il dataset Golub (1999), che contiene l'espressione di 3051 geni in 38 pazienti affetti da due tipi di leucemia; ALL (Leucemia Linfoblastica Acuta) per 27 pazienti e AML (Leucemia Mieloide Acuta) per 11 pazienti.

```
BIC <- mclustBIC(golub_tr)
Bayesian Information Criterion (BIC):
```

	EII	VII	EEI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
2	-207332.3	-206990.8	-188359.2	-188048.3	-188666.8	-187204.6
3	-208654.0	-208514.3	-187536.4	-187523.8	-190728.2	-190329.1
4	-207250.8	-206430.9	-188880.7	-188041.9	-196012.4	-194672.7
5	-211290.1	NA	-190745.2	NA	NA	NA
6	-215322.6	NA	-194599.0	NA	NA	NA
7	-219106.4	NA	-198747.2	NA	NA	NA
8	-223836.1	NA	-202300.4	NA	NA	NA
9	-227666.3	NA	-206199.8	NA	NA	NA

Top 3 models based on the BIC criterion:

	VVI,2	VEI,3	EEI,3
	-187204.6	-187523.8	-187536.4

```
summary(BIC, golub_tr)
```

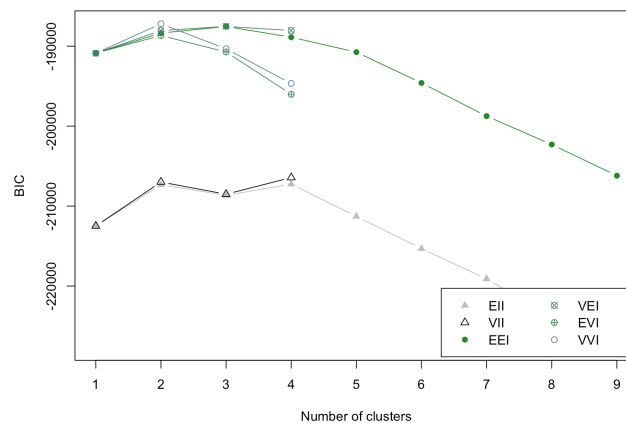
Best BIC values:

	VVI,2	VEI,3	EEI,3
BIC	-187204.6	-187523.7554	-187536.36
BIC diff	0.0	-319.1174	-331.72

Classification table for model (VVI,2):

1	2
27	11

```
plot(BIC, xlab = "Number of clusters")
```



```
k <- seq(from = 1, to = 5, by = 2)
```

```
mclustBIC(golub_tr, G = k)
```

Bayesian Information Criterion (BIC):

	EII	VII	EEI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
3	-208654.0	-208514.3	-187536.4	-187523.8	-190728.2	-190329.1
5	-211290.1	NA	-190745.2	NA	NA	NA

Top 3 models based on the BIC criterion:

	VEI,3	EEI,3	VVI,3
	-187523.8	-187536.4	-190329.1

Nel codice si utilizza la funzione `mclustBIC` per la stima del BIC associata a diversi modelli di misture gaussiane (EII, VII, EEI, VEI, EVI, VVI) e a un diverso numero di cluster (da 1 a 9). Il modello che è risultato il migliore è VVI con 2 cluster (BIC ≈ -187204.6). Seguono VEI con 3 cluster e EEI con 3 cluster, ma con BIC più bassi. La tabella di classificazione per VVI mostra che le 38 unità sono state ripartite nei due gruppi nelle quantità 27 e 11.

Il grafico riporta l'andamento del BIC al variare del numero di cluster. Ogni linea colorata corrisponde a uno dei modelli di mistura (EII, VII, EEI, VEI, EVI, VVI). Si nota che alcuni modelli (ad esempio VVI, VEI, EEI) ottengono valori di BIC meno negativi, quindi migliori, specialmente per un numero di cluster compreso fra 2 e 3.

È stata rifatta l'analisi per un ristretto numero di cluster ($G = 1, 3, 5$) e i tre modelli migliori sono VEI con 3 cluster, EEI con 3 cluster, e VVI con 3 cluster. Si conferma che 3 cluster fornisce un BIC più alto di 1 o 5 cluster, seppure i valori non siano migliori di (VVI,2).

```
hc1 <- hc(golub_tr, modelName = "VVV", use = "SVD")
BIC <- mclustBIC(golub_tr, initialization = list(hcPairs = hc1))
```

	EII	VII	EEI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
2	-217676.9	NA	-193411.4	NA	NA	NA
3	-224928.4	NA	-199606.7	NA	NA	NA
4	-231869.2	NA	-205276.7	NA	NA	NA
5	-238123.3	NA	-210266.2	NA	NA	NA
6	-233565.1	NA	-215832.6	NA	NA	NA
7	-240786.7	NA	-221633.0	NA	NA	NA
8	-247092.1	NA	-227072.3	NA	NA	NA
9	-254496.7	NA	-232544.5	NA	NA	NA

Top 3 models based on the BIC criterion:

EEI,1	EVI,1	VEI,1
-190878.2	-190878.2	-190878.2

```
hc2 <- hc(golub_tr, modelName = "EEE", use = "SVD")
BIC2 <- mclustBIC(golub_tr, initialization = list(hcPairs = hc2))
```

	EII	VII	EEI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
2	-207332.3	-206990.8	-188359.2	-188048.3	-188666.8	-187204.6
3	-208654.0	-208514.3	-187536.4	-187523.8	-190728.2	-190329.1
4	-207250.8	-206430.9	-188880.7	-188041.9	-196012.4	-194672.7
5	-211290.1	NA	-190745.2	NA	NA	NA
6	-215322.6	NA	-194599.0	NA	NA	NA
7	-219106.4	NA	-198747.2	NA	NA	NA
8	-223836.1	NA	-202300.4	NA	NA	NA
9	-227666.3	NA	-206199.8	NA	NA	NA

Top 3 models based on the BIC criterion:

VVI,2	VEI,3	EEI,3
-187204.6	-187523.8	-187536.4

```
BIC3 <- mclustBIC(golub_tr, verbose = FALSE, initialization = list(hcPairs =
hcRandomPairs(golub_tr)))
```

Bayesian Information Criterion (BIC):

	EII	VII	EEI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
2	-207692.0	-207898.7	-189260.0	-189250.1	-203867.8	-203848.1
3	-208326.3	-210520.4	-192809.4	-191872.4	-217897.2	-217744.2
4	-216504.9	-218476.2	-201074.0	-199958.4	-232795.5	-227478.4
5	-219336.9	-226148.2	-203432.7	-206458.7	-249903.8	-237325.6
6	-227792.9	-230990.8	-214158.5	-215013.5	-262849.7	-250563.8
7	-235590.5	-241153.6	-227214.0	-222592.5	-276879.1	-261410.5
8	-243063.5	-248043.5	-234114.0	-229407.1	-291544.1	-271175.7
9	-251896.1	-256077.1	-241464.7	-236287.1	-301084.8	-276114.0

Top 3 models based on the BIC criterion:

VEI,2	EEI,2	EEI,1
-189250.1	-189260.0	-190878.2

```
summary(BIC)
```

Best BIC values:

	EEI,1	EVI,1	VEI,1
BIC	-190878.2	-190878.2	-190878.2
BIC diff	0.0	0.0	0.0

```
summary(BIC2)
```

Best BIC values:

	VVI,2	VEI,3	EEI,3
BIC	-187204.6	-187523.7554	-187536.36
BIC diff	0.0	-319.1174	-331.72

```
summary(BIC3)
Best BIC values:
      VEI,2      EEI,2      EEI,1
BIC    -189250.1 -1.892600e+05 -190878.223
BIC diff      0.0 -9.893477e+00  -1628.078

BICt <- NULL
for (j in 1:100) {
  rBIC <- mclustBIC(golub_tr, verbose = FALSE, initialization = list(hcPairs =
randomPairs(golub_tr)))
  BICt <- mclustBICupdate(BICt, rBIC)
}

summary(BICt)
Best BIC values:
      VEI,2      EEI,2      VEI,3
BIC    -186661 -186685.95300 -186769.7755
BIC diff      0      -24.97411  -108.7966
```

Si crea `hc1` mediante la funzione `hc()` con modello “VVV” e opzione `use = "SVD"`. Poi si calcola il BIC usando come inizializzazione i “pairwise merges” forniti da `hc1`. L’output riporta la matrice BIC per il numero di cluster da 1 a 9 e per i diversi modelli di covarianza. Si evidenziano i tre migliori modelli in base al BIC, e questi sono EEI,1, EVI,1 e VEI,1 (BIC = -190878.2).

Si crea `hc2` con modello “EEE” e si calcola il BIC di nuovo. Le prime tre soluzioni sono VVI,2, VEI,3 e EEI,3, con VVI,2 come migliore (BIC = -187204.6).

Infine si utilizza `hcRandomPairs(golub_tr)` per una inizializzazione casuale. I tre modelli migliori sono VEI,2, EEI,2 e EEI,1, con VEI,2 migliore (BIC = -189250.1). Dal ciclo (100 iterazioni) con inizializzazioni casuali (`randomPairs`) otteniamo invece VEI,2 con BIC $\approx$ -186661, EEI,2 con BIC $\approx$ -186685.95, VEI,3 con BIC $\approx$ -186769.78

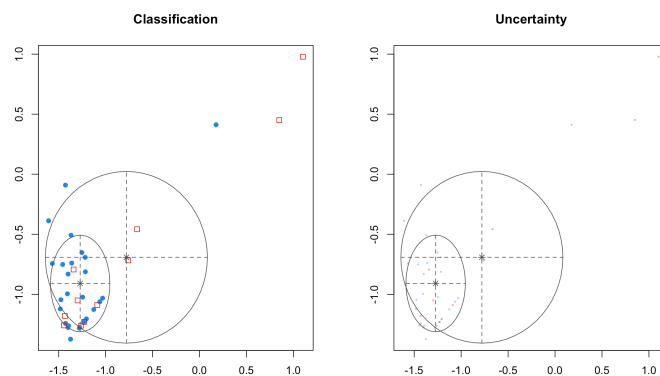
Nel complesso, i modelli “VEI” e “EEI” (con 2 o 3 cluster) emergono spesso come buoni candidati, ma i risultati mostrano come il processo di selezione del modello sia sensibile al metodo iniziale di partizionamento.

```
mod <- Mclust(golub_tr)
round(mod$parameters$mean, 2)
      [,1] [,2]
[1,] -1.27 -0.78
[2,] -0.91 -0.69
[3,]  0.27  0.25
[4,]  1.00  0.83
[5,]  0.69 -0.04

table(mod$classification)
1  2
27 11

modt <- Mclust(golub_tr, x = BICt)

par(mfrow = c(1, 2))
plot(mod, what = "classification", dims = 1:2)
title(main = "Classification")
plot(mod, what = "uncertainty", dims = 1:2)
title(main = "Uncertainty")
summary(mod)
summary(mod, parameters = TRUE)
```



Effettuiamo un clustering non supervisionato cercando il miglior numero di cluster e la migliore parametrizzazione della covarianza, secondo il BIC. Le 38 unità sono suddivise in 27 nel primo cluster e 11 nel secondo cluster.

Il grafico mostra la classificazione e l'incertezza dei punti. A sinistra ogni punto è etichettato in base al cluster assegnato e si osserva il gruppo più numeroso (cluster 1). Le ellissi rappresentano le regioni di confidenza per ciascun cluster; più i punti sono vicini al confine tra cluster (o meno densi), più alta è l'incertezza.

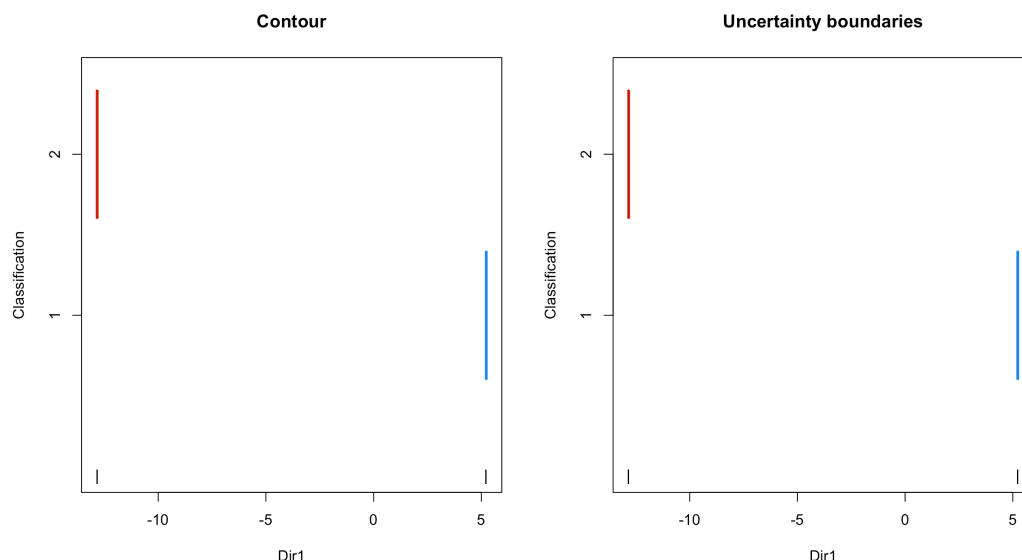
```
table(golub.cl, mod$classification)
golub.cl 1 2
0 27 0
1 0 11

ARI <- adjustedRandIndex(golub.cl, mod$classification)
round(ARI, 2)
[1] 1
```

La tabella di classificazione mostra che 27 osservazioni edichettate con “0” cadono nel cluster 1 e 11 osservazioni etichettate con “1” cadono nel cluster 2; le due classificazioni coincidono. Ciò è confermato dal valore dell’ARI (1). La soluzione ottenuta con il modello mistura risulta identica a quella fornita dai metodi di classificazione basati su distanze.

```
drmod <- MclustDR(mod, lambda = 1)

par(mfrow = c(1, 2))
plot(drmod, what = "contour")
title(main = "Contour")
plot(drmod, what = "boundaries", ngrid = 200)
title(main = "Uncertainty boundaries")
```



Il grafico è una rappresentazione bidimensionale in cui, sull’asse x c’è la proiezione dei dati nello spazio ridotto, mentre l’asse y c’è il cluster di appartenenza. Le linee colorate sono una visualizzazione della distribuzione dei punti nei cluster secondo una sola dimensione. L’assenza di sovrapposizioni suggerisce una buona separabilità delle due classi nella direzione estratta.

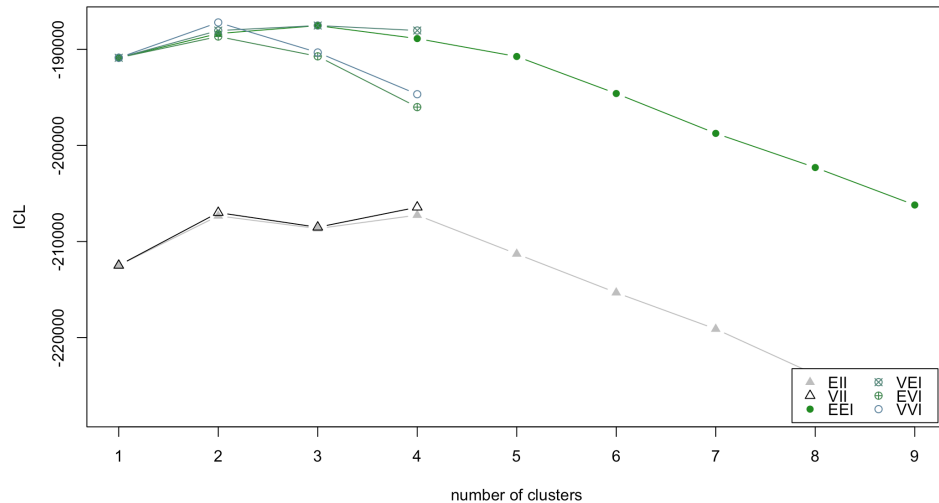
```
ICL <- mclustICL(golub_tr)
Integrated Complete-data Likelihood (ICL) criterion:
```

	EII	VII	EVI	VEI	EVI	VVI
1	-212484.9	-212484.9	-190878.2	-190878.2	-190878.2	-190878.2
2	-207332.3	-206990.8	-188359.2	-188048.3	-188666.8	-187204.6
3	-208654.0	-208514.3	-187536.4	-187523.8	-190728.2	-190329.1
4	-207250.8	-206430.9	-188880.7	-188041.9	-196012.4	-194672.7
5	-211290.1	NA	-190745.2	NA	NA	NA
6	-215322.6	NA	-194599.0	NA	NA	NA
7	-219106.4	NA	-198747.2	NA	NA	NA
8	-223836.1	NA	-202300.4	NA	NA	NA
9	-227666.3	NA	-206199.8	NA	NA	NA

Top 3 models based on the ICL criterion:

VVI,2 VEI,3 EEI,3
-187204.6 -187523.8 -187536.4

```
plot(ICL, xlab = "number of clusters")
```



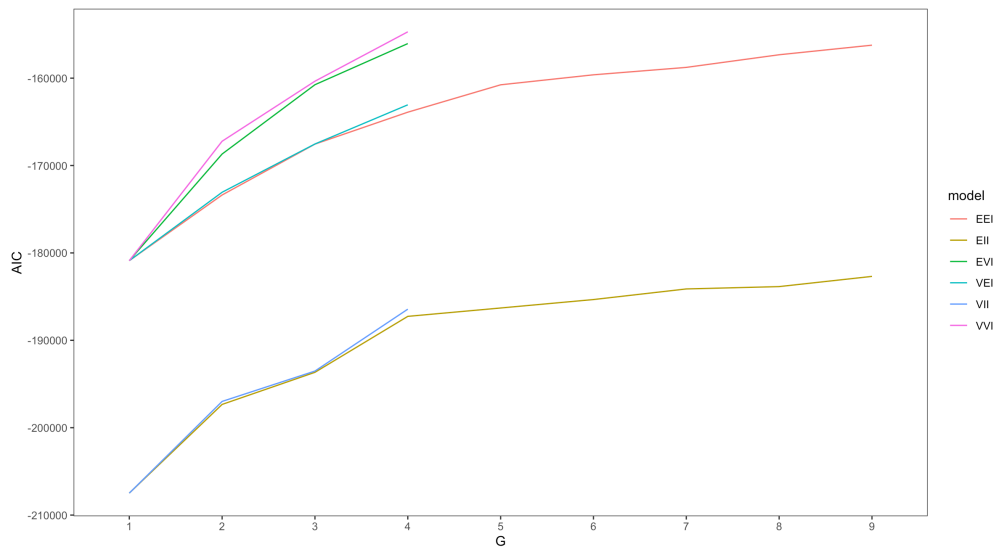
Il criterio ICL (Integrated Complete-data Likelihood) è un'estensione del BIC che penalizza la sovrapposizione tra i cluster. L'ICL privilegia le soluzioni in cui i cluster siano più nettamente separati. Le prime tre soluzioni individuate dal criterio sono VVI con 2 cluster (ICL = -187204.6), VEI con 3 cluster (ICL = -187523.8) e EEI con 3 cluster (ICL = -187536.4). Nel grafico si disegna l'andamento dei valori di ICL in funzione del numero di cluster. Ogni linea rappresenta un diverso modello di covarianza. Si nota che le curve relative ad alcuni modelli rimangono più in alto quando il numero di cluster è 2 o 3.

```
library(ggplot2)
G <- attr(BIC, "G")
modelN <- attr(BIC, "modelNames")
d <- attr(BIC, "d")
criterio <- data.frame(
  G = rep(G, length(modelN)),
  model = rep(modelN, each = length(G)),
  AIC = NA,
  stringsAsFactors = FALSE
)

for (i in 1:nrow(criterio)) {
  m <- Mclust(golub_tr, G = criterio$G[i], modelName = criterio$model[i])
  if (!is.null(m)) {
    p <- nMclustParams(criterio$model[i], d, criterio$G[i])
    criterio[i, "AIC"] <- 2 * m$loglik - 2 * p
  }
}

head(criterio[order(criterio$AIC, decreasing = TRUE), ], 3)

ggplot(criterio, aes(G, AIC, color = model)) +
  geom_line() +
  scale_x_discrete(limits = G) +
  theme_bw() +
  theme(
    panel.grid.major.x = element_blank(),
    panel.grid.major.y = element_blank(),
    panel.grid.minor.y = element_blank()
  )
```



Si calcola l'AIC per ciascun modello mistura di gaussiane e per un numero di cluster variabile da 1 a 9.

Sull'asse x abbiamo il numero di cluster da 1 a 9, sull'asse y abbiamo il valore dell'AIC associato a ciascuna combinazione di (modello, cluster). Valori più alti di AIC indicano una qualità migliore della partizione. Le linee colorate corrispondono ai diversi modelli di covarianza. Le tre migliori combinazioni di numero di cluster e modello in termini di AIC sono VVI con 4 cluster ($AIC \approx -154697.6$) (linea rosa), EVI con 4 cluster ($AIC \approx -156042.2$) (linea verde), EEI con 9 cluster ($AIC \approx -156223.9$) (linea rossa).